

MVP Requirements

Обновлено 13 мая 2026, 21:19 Андрей Коновалов

Mini Tournament Hub - MVP Requirements

1. Общая информация

Mini Tournament Hub - backend для коротких live-турниров, где участники отвечают текстом на случайные раундовые prompts, затем последовательно голосуют за ответы других участников. MVP покрывает один режим: multi-round tournament without elimination.

Сквозной сценарий MVP:

1. Пользователь регистрируется или входит в систему.
2. Пользователь создает public или private турнир.
3. Backend автоматически добавляет владельца в участники.
4. Участники присоединяются к draft-турниру, для private требуется inviteToken.
5. Владелец запускает турнир при наличии минимум 4 участников.
6. Backend создает первый раунд, назначает persisted bilingual text prompt и открывает SUBMISSION.
7. Активные участники отправляют или обновляют один текстовый submission.
8. Submission phase завершается по дедлайну или когда все активные подключенные участники отправили ответы.
9. Backend переводит раунд в VOTING и последовательно раскрывает submissions по одному.
10. Участники голосуют LIKE или DISLIKE за текущий раскрытый submission, кроме собственного.
11. Для каждого submission голосование завершается по дедлайну или когда все активные eligible voters проголосовали.
12. После всех submissions backend фиксирует результаты раунда, обновляет cumulative leaderboard и создает следующий раунд либо завершает турнир.
13. Клиенты получают live-обновления через [Socket.IO](#) и восстанавливают состояние через REST snapshot.

2. Scope MVP

2.1. Входит в MVP

- JWT auth: registration, login, refresh, logout.
- Создание public/private турнира.

- inviteToken для private-турниров.
- Автоматическое добавление owner в tournament_participants.
- Ограничение: owner может иметь только один unfinished tournament (DRAFT или ACTIVE).
- Draft cancellation владельцем.
- Join/leave турнира через REST, leave разрешен только в DRAFT и не для owner.
- Ограничение активного участия: пользователь не должен одновременно участвовать в нескольких ACTIVE турнирах.
- roundsCount от 1 до 4.
- submissionDurationSeconds и voteDurationSeconds: только 15, 30 или 45 секунд.
- Minimum participants to start: 4.
- Persisted text prompt для каждого раунда, с en и ru контентом.
- Один текстовый submission на участника в рамках раунда, upsert до закрытия submission phase.
- Последовательное voting по одному раскрытому submission.
- Manual votes LIKE/DISLIKE, upsert до закрытия текущего voting window.
- Auto-timeout votes: отсутствующие голоса eligible participants фиксируются как DISLIKE при дедлайне.
- Round score равен likeCount; dislikeCount хранится и отдается в событиях, но не вычитается из score.
- Cumulative score участников между раундами.
- Автоматическое создание следующего раунда.
- Автоматическое завершение турнира после последнего раунда.
- Live updates через [Socket.IO](#) rooms tournament:{tournamentId}.
- Redis-backed presence tracking с multi-tab semantics.
- REST recovery endpoints: GET /api/v1/tournaments/:id/full и GET /api/v1/users/me/live-tournament.
- Swagger /api и AsyncAPI /api-ws, если включены настройками.

2.2. Не входит в MVP

- Несколько tournament modes.
- Неформатированный пользовательский контент, загрузка файлов, изображения, мемы и URL-content как отдельные типы.
- Редактируемые submission rules сверх server-side text validation.
- Spectator mode.
- Public room access для неучастников.
- Chat, comments, reactions.

- Admin/moderator roles.
- Anti-fraud.
- Notifications.
- Совместное управление турниром.
- Редактирование ключевых параметров после создания/запуска.
- Distributed **Socket.IO** adapter.
- Event replay/history/event sourcing.
- BullMQ/cron-based phase orchestration.
- Сложная аналитика и отдельная история действий.

3. Термины и сущности

Tournament

Поля в текущем коде: title, description, visibility, roundsCount, submissionDurationSeconds, voteDurationSeconds, status, inviteToken, ownerId, winnerId.

Statuses: DRAFT, ACTIVE, COMPLETED, CANCELLED.

Participant

Строка tournament_participants с composite key { tournamentId, userId }, cumulativeScore, createdAt, updatedAt. createdAt используется как deterministic tie-break при равном score.

Round

Раунд хранит number, phase, prompt fields, submissionDeadline, submissionClosedAt, current voting state, votingDeadline, votingCompletedAt, completedAt.

Round phase enum: SUBMISSION, VOTING.

Voting step status: IDLE, OPEN, FINALIZING, FINISHED.

Prompt

Persisted bilingual text prompt:

json

```
{ "key": "string", "type": "TEXT", "content": { "en": "string", "ru": "string" } }
```

Prompt выбирается случайно без повторов в рамках одного турнира.

Submission

Текстовый ответ участника в конкретном раунде. Один author может иметь не более одного submission на round. Поля: roundId, authorId, content, submittedAt, likeCount, dislikeCount, roundScore.

Vote

Голос участника за текущий раскрытый submission. Поля: roundId, submissionId, voterId, value, source, votedAt.

value: LIKE или DISLIKE.

source: manual vote или timeout-generated vote.

Presence

Ephemeral Redis-backed state активных socket-подключений. Presence не является фактом участия в турнире и не сохраняется как domain history.

4. Роли пользователей

Авторизованный пользователь

- Создает турнир, если не владеет другим unfinished tournament.
- Присоединяется к доступному турниру через REST.
- Получает full state только если является participant.
- Подключается к WebSocket room только если является participant.

Участник турнира

- Отправляет submission в SUBMISSION.
- Голосует в VOTING.
- Может покинуть турнир только пока он DRAFT и если пользователь не owner.

- Может участвовать только в одном ACTIVE турнире.

Владелец турнира

- Настраивает параметры при создании.
- Запускает DRAFT tournament.
- Отменяет DRAFT tournament.
- Не может покинуть собственный tournament через leave.

5. Бизнес-правила

BR-1. Поддерживается только один режим: multi-round text-prompt tournament without elimination.

BR-2. roundsCount фиксируется при создании и должен быть от 1 до 4.

BR-3. Участники не выбывают между раундами.

BR-4. Tournament winner определяется после завершения последнего раунда.

BR-5. В каждый момент времени у active tournament должен быть один current round: незавершенный round с completedAt = null.

BR-6. Round phases идут строго: SUBMISSION -> VOTING -> completedAt.

BR-7. Один participant может иметь не более одного submission в рамках round; повторная отправка обновляет существующий submission.

BR-8. Submission content - строка длиной 1..4000.

BR-9. Submission нельзя создать или изменить после submissionDeadline, submissionClosedAt или перехода из SUBMISSION.

BR-10. Owner автоматически является participant.

BR-11. Старт разрешен только owner, только из DRAFT, только при минимум 4 participants.

BR-12. Owner не может создать второй DRAFT или ACTIVE tournament.

BR-13. Пользователь не может быть participant в двух ACTIVE tournaments одновременно.

BR-14. Draft participation допускается даже если пользователь участвует в другом draft.

BR-15. Private tournament join требует валидный inviteToken.

BR-16. COMPLETED и CANCELLED tournaments недоступны для join.

BR-17. Draft tournament может быть cancelled только owner; cancellation удаляет participant rows и eject'ит sockets из room.

BR-18. WebSocket client messages являются только transport actions: tournament:join, tournament:leave.

BR-19. Domain mutations выполняются через REST.

BR-20. Room access в MVP только для participants, без spectators.

BR-21. During voting backend раскрывает один current submission за раз.

BR-22. Vote разрешен только participant'у target tournament, только в VOTING, только когда voting step OPEN, только за current revealed submission.

BR-23. Author не может голосовать за собственный submission.

BR-24. Vote можно изменить до истечения current voting window.

BR-25. При истечении voting window missing votes eligible participants фиксируются как DISLIKE.

BR-26. Round score submission равен likeCount.

BR-27. Cumulative score participant увеличивается на roundScore его submissions.

BR-28. Leaderboard сортируется по cumulativeScore DESC, затем participant.createdAt ASC, затем userId ASC.

BR-29. Winner tournament - первый participant из итогового leaderboard; отдельный tie-break round не создается.

BR-30. История WebSocket events не хранится; recovery выполняется через REST snapshot.

6. Функциональные требования

6.1. Auth

FR-1. POST /api/v1/auth/register создает пользователя по email, username, password и возвращает JWT pair.

FR-2. POST /api/v1/auth/login выполняет вход по email, password и возвращает JWT pair.

FR-3. POST /api/v1/auth/refresh обновляет JWT pair по refresh token.

FR-4. DELETE /api/v1/auth/logout удаляет refresh token текущего fingerprint.

FR-5. Access-protected endpoints используют JWT Bearer token.

6.2. Tournament Management

FR-6. POST /api/v1/tournaments создает tournament с полями:

- title
- description
- visibility
- roundsCount
- submissionDurationSeconds
- voteDurationSeconds

FR-7. Для private tournament backend генерирует inviteToken.

FR-8. Owner автоматически добавляется в participants.

FR-9. POST /api/v1/tournaments/:id/join добавляет пользователя в participants, для private требует body inviteToken.

FR-10. Repeated join для уже существующего participant должен быть idempotent.

FR-11. POST /api/v1/tournaments/:id/leave удаляет participant только в DRAFT; owner leave запрещен.

FR-12. POST /api/v1/tournaments/:id/cancel переводит draft tournament в CANCELLED, удаляет participants и отправляет realtime event.

FR-13. POST /api/v1/tournaments/:id/start переводит tournament в ACTIVE, создает round 1 и отправляет tournament:started + round:created.

FR-14. Start должен проверить, что ни один participant не участвует в другом active tournament.

6.3. Round And Submission

FR-15. При создании round backend сохраняет prompt fields в tournament_rounds.

FR-16. POST /api/v1/rounds/:roundId/submissions создает или обновляет submission текущего пользователя.

FR-17. После успешного submission backend отправляет round:progress_updated только со счетчиками, без submission content.

FR-18. Submission phase завершается:

- по submissionDeadline;
- или досрочно, если submittedCount >= totalActiveParticipants и totalActiveParticipants > 0.

FR-19. При завершении submission phase backend выставляет phase = VOTING, submissionClosedAt, отправляет round:phase_changed и запускает sequential voting.

6.4. Voting

FR-20. POST /api/v1/rounds/:roundId/votes создает или обновляет manual vote за текущий раскрытый submission.

FR-21. Body vote endpoint:

- submissionId
- value: LIKE или DISLIKE

FR-22. Backend должен отклонять vote, если:

- round не найден или не в VOTING;
- voting step не OPEN;
- submissionId не равен currentVotingSubmissionId;
- votingDeadline истек;
- voter является author submission;
- voter не participant tournament.

FR-23. После manual vote backend отправляет vote:progress_updated.

FR-24. Current submission voting завершается:

- по votingDeadline;
- или досрочно, когда все active eligible voters проголосовали.

FR-25. При финализации current submission backend добавляет missing timeout votes как DISLIKE, считает likeCount/dislikeCount, отправляет vote:finalized и раскрывает следующий submission либо завершает round.

6.5. Round Completion And Tournament Completion

FR-26. После финализации всех submissions backend:

- считает round results;
- сохраняет likeCount, dislikeCount, roundScore в submissions;
- обновляет tournament_participants.cumulativeScore;
- выставляет round.completedAt;
- отправляет round:completed.

FR-27. Если round не последний, backend создает следующий SUBMISSION round, назначает prompt, schedule'ит deadline и отправляет round:created.

FR-28. Если round последний, backend переводит tournament в COMPLETED, сохраняет winnerId и отправляет tournament:finished.

6.6. Recovery And Snapshots

FR-29. GET /api/v1/tournaments/:id/full возвращает current tournament snapshot только для participants.

Текущий implemented response содержит:

- tournament id/title/description/visibility/status;
- roundsCount;
- phase durations;
- ownerId;
- participants с userId и cumulativeScore;
- current round с id, number, phase, prompt, submissionDeadline, submissionClosedAt, votingDeadline.

FR-30. GET /api/v1/users/me/live-tournament возвращает 200:

json

```
{ "hasActiveTournament": true, "tournament": { "id": "uuid", "title": "Spring Cup", "status": "ACTIVE", "roundId": "uuid", "roundNumber": 2, "phase": "VOTING" } }
```

или:

json

```
{ "hasActiveTournament": false, "tournament": null }
```

7. API Requirements

[API-1](#). Все REST payloads передаются в JSON.

[API-2](#). Все protected endpoints используют JWT Bearer token.

[API-3](#). Responses проходят через общий response interceptor.

[API-4](#). Ошибки возвращаются через global exception filter с HTTP status code и message.

[API-5](#). Swagger доступен на /api, если enabled.

[API-6](#). AsyncAPI доступен на /api-ws, если enabled.

7.1. Implemented REST endpoints

text

```
POST /api/v1/auth/register POST /api/v1/auth/login POST /api/v1/auth/refresh DELETE /api/v1/auth/logout GET /api/v1/users/profile GET /api/v1/users/me/live-tournament POST /api/v1/tournaments POST /api/v1/tournaments/:id/join GET /api/v1/tournaments/:id/full POST /api/v1/tournaments/:id/start POST /api/v1/tournaments/:id/leave POST /api/v1/tournaments/:id/cancel POST /api/v1/rounds/:roundId/submissions POST /api/v1/rounds/:roundId/votes
```

7.2. Not implemented in current backend

Эти endpoints были в раннем requirements draft, но сейчас отсутствуют в контроллерах:

text

```
GET /api/v1/tournaments GET /api/v1/tournaments/:id GET /api/v1/tournaments/:id/participants GET /api/v1/rounds/:id PATCH /api/v1/rounds/:id/submissions/:submissionId PATCH /api/v1/rounds/:id/votes/:voteId GET /api/v1/tournaments/:id/results
```

Upsert semantics сейчас реализованы через POST /rounds/:roundId/submissions и POST /rounds/:roundId/votes.

8. WebSocket Requirements

[WS-1](#). Transport: [Socket.IO](#) via [platform-socket.io](#).

[WS-2](#). Auth: access JWT передается в [Socket.IO](#) handshake auth.token.

[WS-3](#). Invalid socket auth приводит к immediate disconnect.

WS-4. Client-to-server socket messages:

- tournament:join
- tournament:leave

WS-5. Socket messages не выполняют domain mutations.

WS-6. Room format: tournament:{tournamentId}.

WS-7. Room join разрешен только tournament participants.

WS-8. Presence хранится в Redis как unique online users per tournament, multi-tab safe.

WS-9. Disconnect не является domain leave и не отправляет tournament:participant_left.

WS-10. Server-to-client events:

- tournament:participant_joined
- tournament:participant_left
- tournament:started
- round:created
- round:phase_changed
- tournament:presence_updated
- round:progress_updated
- voting:submission_revealed
- vote:progress_updated
- vote:finalized
- round:completed
- tournament:finished
- tournament:cancelled

WS-11. tournament:presence_updated payload intentionally lightweight:

json

```
{ "tournamentId": "uuid", "activeCount": 3, "occurredAt": "2026-05-13T12:00:00.000Z" }
```

[WS-12](#). round:progress_updated не содержит submission content.

[WS-13](#). voting:submission_revealed содержит один current submission и votingDeadline.

[WS-14](#). vote:finalized относится к одному submission, а не ко всему round.

[WS-15](#). round:completed содержит rankings и cumulative leaderboard.

[WS-16](#). WebSocket history не хранится и не replay'ится.

[WS-17](#). Reconnect flow:

1. socket reconnect with JWT;
2. client emits tournament:join;
3. client calls GET /api/v1/tournaments/:id/full;
4. UI renders authoritative snapshot.

9. Non-Functional Requirements

[NFR-1](#). Backend реализован на NestJS.

[NFR-2](#). Persistence: PostgreSQL через TypeORM.

[NFR-3](#). Cache/presence/session support: Redis через ioredis wrapper.

[NFR-4](#). Passwords хранятся только в hashed виде.

[NFR-5](#). DTO validation выполняется через class-validator и global ValidationPipe.

[NFR-6](#). Business logic находится в services, controllers остаются thin.

[NFR-7](#). Phase timers в MVP in-memory, с восстановлением открытых rounds on application bootstrap.

[NFR-8](#). MVP рассчитан на один backend instance.

[NFR-9](#). Redis presence не заменяет distributed [Socket.IO](#) adapter.

[NFR-10](#). Event history не хранится.

10. Lifecycle

Tournament lifecycle

text

DRAFT -> ACTIVE -> COMPLETED | -> CANCELLED

Round lifecycle

text

SUBMISSION -> VOTING / IDLE -> VOTING / OPEN for submission #1 -> VOTING / FINALIZING -> VOTING / OPEN for submission #N -> VOTING / FINISHED -> completedAt set

11. Acceptance Criteria

AC-1. Пользователь может зарегистрироваться, войти, refresh token и logout.

AC-2. Пользователь может создать public/private tournament, если не владеет unfinished tournament.

AC-3. Private tournament получает inviteToken.

AC-4. Owner автоматически становится participant.

AC-5. Participant может join draft/private tournament с валидным invite token.

AC-6. Owner может cancel draft tournament.

AC-7. Owner не может leave собственный tournament.

AC-8. Non-owner participant может leave только DRAFT.

AC-9. Owner может start tournament только при минимум 4 participants.

AC-10. Start создает first round с persisted prompt и submission deadline.

AC-11. Round prompt возвращается в round:created и GET /tournaments/:id/full.

AC-12. Participant может создать или обновить один text submission в SUBMISSION.

AC-13. Submission content не транслируется во время submission phase.

- AC-14. Submission phase завершается по deadline или по отправке всеми active participants.
- AC-15. Voting раскрывает submissions последовательно по одному.
- AC-16. Participant может LIKE/DISLIKE current revealed submission.
- AC-17. Author не может голосовать за собственный submission.
- AC-18. Vote можно изменить до закрытия current voting window.
- AC-19. Missing votes на deadline становятся timeout DISLIKE.
- AC-20. vote:finalized содержит likeCount и dislikeCount current submission.
- AC-21. round:completed содержит rankings и cumulative leaderboard.
- AC-22. Если есть следующий round, backend создает его автоматически.
- AC-23. После последнего round backend выставляет tournament COMPLETED и сохраняет winnerId.
- AC-24. Winner определяется по cumulative score, затем по participant creation time, затем по userId.
- AC-25. Socket disconnect не удаляет participant и не отправляет domain leave event.
- AC-26. Reconnect recovery работает через GET /api/v1/tournaments/:id/full.
- AC-27. GET /api/v1/users/me/live-tournament возвращает stable 200 response с active tournament summary или null.

12. Допущения

- Пользователи ведут себя добросовестно.
- MVP работает на одном backend instance.
- Текущего snapshot достаточно для frontend recovery.
- In-memory deadline registries приемлемы для MVP.
- Redis используется для presence, но не для [Socket.IO](#) horizontal scaling.
- Tie-break по суммарному времени прохождения не реализован в текущей модели данных.

13. Риски

- Потеря in-memory timers при restart между bootstrap recovery и дедлайном.
- Race conditions при одновременных votes и phase finalization.
- Room access service все еще проверяет unfinished participation при socket join; это может конфликтовать с draft-multi-participation policy.
- Full tournament snapshot сейчас минимален и не содержит всю историю rounds/submissions/results/winner.
- participant.createdAt является join-time tie-break, но не отдельным first connection timestamp.
- Auto-timeout DISLIKE может учитывать offline participants при deadline finalization.